```
Start coding or generate with AI.
```

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
# Create the dataset
data = [('Buy one get one free. Shop now and save big.', 'Promotional'), ('The attendance report will be finalized this wee

df = pd.DataFrame(data, columns=["message", "label"])

print("Dataset shape:", df.shape)
display(df.head(56))
```

```
Dataset shape: (56, 2)
```

	message	label
0	Buy one get one free. Shop now and save big.	Promotional
1	The attendance report will be finalized this w...	Important
2	Reminder: the coding contest closes tonight, r...	Important
3	The classroom for today's lecture has changed ...	Important
4	The placement aptitude test is scheduled for S...	Important
5	Students selected for the internship should re...	Important

```
# Inspect the dataset
print("Class distribution:")
display(df["label"].value_counts())

print("\nMissing values:")
print(df.isnull().sum())

# Visualize class distribution
df["label"].value_counts().plot(kind="bar", rot=0, figsize=(6,4))
plt.title("Number of Messages in Each Class")
plt.xlabel("Class")
plt.ylabel("Number of Messages")
plt.show()
```

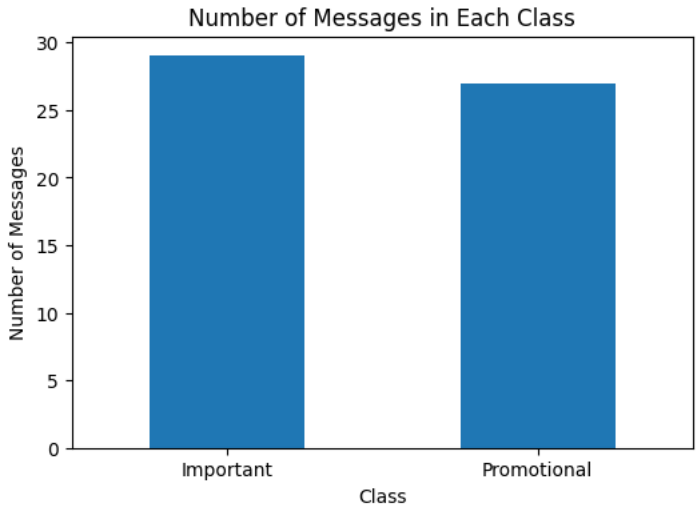
	message	label
15	Your exclusive bonus is ready. Click now to cl...	Promotional
16	Your internal assessment marks are available o...	Important
17	The timetable for the next semester is now ava...	Important
18	...ber has won a shopping voucher.	Promotional
19	Bring your laptop for tomorrow's Python practi...	Important
20	Exclusive offer! Get a free gift voucher today.	Promotional
21	Students must submit the project synopsis by W...	Important
22	Unlock your exclusive coupon and save money to...	Promotional
23	Your project presentation slot is at 10:30 AM.	Important

```
Class distribution:
```

label	count
Important	29
Promotional	27

dtype: int64

```
Missing values:
message    0
label      0
dtype: int64
```




```
# Separate input (X) and target (y)
X = df["message"]
y = df["label"]

# Split before fitting the vectorizer to avoid data leakage
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

print("Training messages:", len(X_train))
print("Testing messages :", len(X_test))
```

```
Training messages: 42
Testing messages: 14
```

48	Download the question bank from the LMS before...	Important
49	Please check your university email for the int...	Important

```
# Convert text into Bag-of-Words features
vectorizer = CountVectorizer(
    lowercase=True,
    stop_words="english"
)

X_train_bow = vectorizer.fit_transform(X_train)
X_test_bow = vectorizer.transform(X_test)

print("Training matrix shape:", X_train_bow.shape)
print("Testing matrix shape :", X_test_bow.shape)

print("\nFirst 30 vocabulary words:")
print(vectorizer.get_feature_names_out()[:30])
```

```
Training matrix shape: (42, 157)
Testing matrix shape : (14, 157)

First 30 vocabulary words:
['10' '11' '204' '30' '70' 'achievement' 'activate' 'ai' 'aptitude'
 'assessment' 'assignment' 'attendance' 'available' 'bank' 'big' 'biggest'
 'bonus' 'brand' 'bring' 'buy' 'campus' 'card' 'cashback' 'certificate'
 'chance' 'changed' 'check' 'chosen' 'claim' 'class']
```

```
# See what Bag-of-Words actually looks like
bow_sample = pd.DataFrame(
    X_train_bow[:5].toarray(),
    columns=vectorizer.get_feature_names_out()
)

display(bow_sample)
```

	10	11	204	30	70	achievement	activate	ai	aptitude	assessment	...	uploaded	users	viva	voucher	waiting	wednesd
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	0	0	...	0	1	0	0	0	

5 rows × 157 columns

```
# Create and train the Naive Bayes classifier
model = MultinomialNB()

model.fit(X_train_bow, y_train)

print("Naive Bayes model trained successfully!")
```

```
Naive Bayes model trained successfully!
```

```
# Make predictions
y_pred = model.predict(X_test_bow)

print("Predictions:")
comparison = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred,
    "Message": X_test.values
})

display(comparison)
```

Predictions:

	Actual	Predicted	Message
0	Important	Important	Please confirm your participation in the works...
1	Important	Promotional	The timetable for the next semester is now ava...
2	Promotional	Promotional	Get 80 percent off on shoes. Limited time offer!
3	Promotional	Promotional	Act now to receive your complimentary gift.
4	Important	Important	The NLP practical will be conducted in Lab 3 t...
5	Important	Important	The university exam timetable has been published.
6	Important	Important	The department meeting for student representat...
7	Promotional	Important	Congratulations winner! Send your details to r...
8	Promotional	Promotional	Unlock your exclusive coupon and save money to...
9	Promotional	Promotional	Get instant cashback on your next purchase.
10	Promotional	Promotional	Earn easy money from home. Join our program to...
11	Promotional	Promotional	Congratulations! You won a free smartphone. Cl...
12	Important	Important	The project review is scheduled for Monday mor...
13	Important	Important	The coding contest registration closes tonight.

Next steps: Generate code with comparison | New interactive sheet

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2%}")

# Detailed metrics
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred, labels=["Important", "Promotional"])

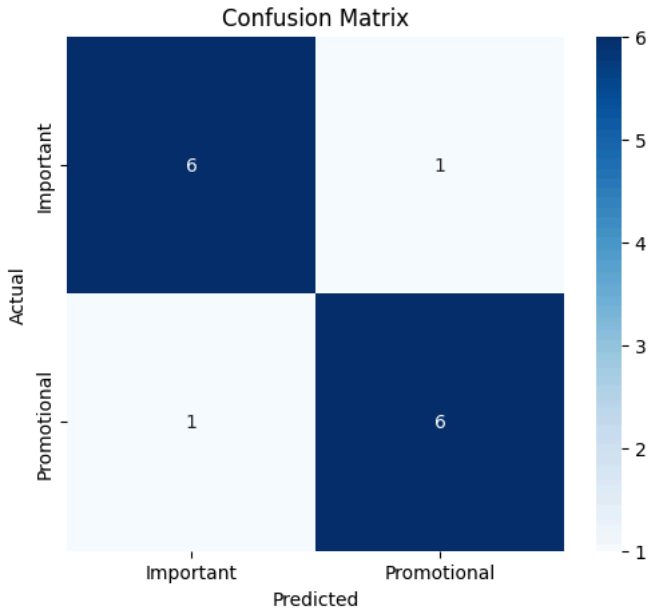
plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Important", "Promotional"],
    yticklabels=["Important", "Promotional"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```
Accuracy: 85.71%

Classification Report:
              precision    recall  f1-score   support

   Important       0.86      0.86      0.86         7
 Promotional       0.86      0.86      0.86         7

    accuracy                           0.86        14
   macro avg       0.86      0.86      0.86        14
weighted avg       0.86      0.86      0.86        14
```




```
new_messages = [
    "Please submit your machine learning assignment by tomorrow.",
    "Congratulations! Claim your free gift now.",
    "The placement aptitude test starts at 9 AM on Saturday.",
    "Win a brand new phone by clicking this link.",
    "The seminar hall has been changed to Room 301.",
    "Get 70 percent discount today. Limited offer!",
    "Your project viva is scheduled for Monday.",
    "You have won an exclusive cashback reward."
]

new_bow = vectorizer.transform(new_messages)
new_predictions = model.predict(new_bow)
new_probabilities = model.predict_proba(new_bow)

results = pd.DataFrame({
    "Message": new_messages,
    "Prediction": new_predictions,
    "Confidence": new_probabilities.max(axis=1)
})

display(results)
```

	Message	Prediction	Confidence
0	Please submit your machine learning assignment...	Important	0.992164
1	Congratulations! Claim your free gift now.	Promotional	0.981202
2	The placement aptitude test starts at 9 AM on ...	Important	0.933639
3	Win a brand new phone by clicking this link.	Promotional	0.986447
4	The seminar hall has been changed to Room 301.	Important	0.956654
5	Get 70 percent discount today. Limited offer!	Promotional	0.995640
6	Your project viva is scheduled for Monday.	Important	0.920259
7	You have won an exclusive cashback reward.	Promotional	0.981202

Next steps: Generate code with `results` New interactive sheet

```
# Inspect important words for each class
feature_names = vectorizer.get_feature_names_out()

class_names = model.classes_

for class_index, class_name in enumerate(class_names):
    log_probs = model.feature_log_prob_[class_index]
    top_indices = np.argsort(log_probs)[-10:][::-1]

    print(f"\nTop words associated with: {class_name}")
    for idx in top_indices:
        print(f"{feature_names[idx]:15s} {log_probs[idx]:.3f}")
```

```
Top words associated with: Important
pm              -3.821
students        -4.227
tomorrow        -4.227
lecture         -4.227
submit          -4.515
report          -4.515
office          -4.515
practical       -4.515
portal          -4.515
seminar         -4.515

Top words associated with: Promotional
free            -3.777
special         -3.959
win             -4.182
reward          -4.182
offer           -4.182
sale            -4.182
discount        -4.182
claim           -4.182
click           -4.182
shop            -4.470
```

```
# Take message from the user
new_message = input("Enter your message: ")

# Convert the message into the same Bag-of-Words representation
new_message_vector = vectorizer.transform([new_message])

# Make prediction
prediction = model.predict(new_message_vector)[0]

# Get prediction probabilities
probabilities = model.predict_proba(new_message_vector)[0]

# Get class names
classes = model.classes_

# Display result
print("\n" + "=" * 50)
print("              PREDICTION RESULT")
print("=" * 50)

print(f"\nMessage: {new_message}")

if prediction == "Important":
    print("\n Prediction: IMPORTANT")
else:
    print("\n Prediction: PROMOTIONAL")

# Display probabilities
print("\nProbability:")
for class_name, probability in zip(classes, probabilities):
    print(f"   {class_name}: {probability:.2%}")

print(f"\nModel Confidence: {max(probabilities):.2%}")
print("=" * 50)
```

```
Enter your message: Tomorrow Last Date to Submission Your Assignment

==================================================
              PREDICTION RESULT
==================================================

Message: Tomorrow Last Date to Submission Your Assignment

 Prediction: IMPORTANT
```

```
Probability:
   Important: 88.95%
   Promotional: 11.05%

Model Confidence: 88.95%
==================================================
```

```
import joblib

# Save trained model
joblib.dump(model, "naive_bayes_model.pkl")

# Save text vectorizer
joblib.dump(vectorizer, "vectorizer.pkl")

print("Model and vectorizer saved successfully!")
```

```
Model and vectorizer saved successfully!
```

```
from flask import Flask, render_template, request
import joblib
import os

app = Flask(__name__)

# Load the trained model and vectorizer
model = joblib.load("naive_bayes_model.pkl")
vectorizer = joblib.load("vectorizer.pkl")


@app.route("/", methods=["GET", "POST"])
def home():
    prediction = None
    message = ""

    if request.method == "POST":
        message = request.form.get("message", "").strip()

        if message:
            # Convert message into numerical features
            message_vector = vectorizer.transform([message])

            # Predict message category
            prediction = model.predict(message_vector)[0]

    return render_template(
        "index.html",
        prediction=prediction,
        message=message
    )


if __name__ == "__main__":
    app.run(debug=True)
```

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Message Classifier</title>

    <style>
        * {
            box-sizing: border-box;
        }

        body {
            font-family: Arial, sans-serif;
            background: #f0f4f8;
            margin: 0;
            padding: 0;
        }

        header {
            background: #173b70;
            color: white;
            text-align: center;
            padding: 25px 10px;
        }
```

```
}

header h1 {
    margin: 0 0 10px;
}

header p {
    margin: 0;
}

.container {
    max-width: 650px;
    margin: 45px auto;
    padding: 30px;
    background: white;
    border-radius: 12px;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

h2 {
    color: #173b70;
    text-align: center;
}

label {
    display: block;
    margin: 20px 0 10px;
    font-weight: bold;
}

textarea {
    width: 100%;
    height: 140px;
    padding: 12px;
    font-size: 16px;
    border: 1px solid #ccc;
    border-radius: 6px;
    resize: vertical;
}

textarea:focus {
    outline: none;
    border-color: #173b70;
}

button {
    width: 100%;
    padding: 13px;
    margin-top: 18px;
    background: #173b70;
    color: white;
    font-size: 16px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
}

button:hover {
    background: #24549a;
}

.result {
    margin-top: 25px;
    padding: 20px;
    border-radius: 8px;
    text-align: center;
}

.important {
    background: #d4edda;
    color: #155724;
    border: 1px solid #b7dfc2;
}

.promotional {
    background: #fff3cd;
    color: #856404;
    border: 1px solid #ffe69c;
}

.result h3 {
    margin-top: 0;
}
```

```
        .classified-message {
            overflow-wrap: anywhere;
        }

        .error {
            color: #b00020;
            margin-top: 15px;
        }

        footer {
            text-align: center;
            padding: 20px;
            color: #555;
        }

        @media (max-width: 600px) {
            .container {
                margin: 25px 15px;
                padding: 20px;
            }
        }
    </style>
</head>

<body>

    <header>
        <h1>Message Classification System</h1>
        <p>Naive Bayes Machine Learning Project</p>
    </header>

    <div class="container">

        <h2>Classify Your Message</h2>

        <p>
            Enter a message below to determine whether it is
            Important or Promotional.
        </p>

        <form method="POST" action="/">

            <label for="message">Enter Your Message:</label>

            <textarea
                id="message"
```